

Epistemology for Knowledge Work in the LLM Era

A second chance for knowledge management, and an answer to the company-brain trend

By François Bouet, August 19, 2026 version

This document is about knowledge work in the LLM era. So much has been done for AI-assisted coding, but for the wider spectrum of knowledge work it's still the Wild West.¹ By knowledge work I mean researching a topic, synthesizing sources, producing documents, and keeping order and control over the accumulation of information. What follows is a vision for staying productive and sane in the face of the volume AI produces. It is also about solving for the anxiety people feel in this line of work, and the sense of being threatened by AI. I am proposing new patterns and tools that allow humans to focus on thinking while the LLM does the housekeeping.

New technologies often bring new ways of managing data: spreadsheets, relational databases, cloud computing, big data, machine learning, search engines, and now LLMs. But this time, it's personal. The data is our own thinking, our bread-and-butter working data, and software can finally read it semantically. The stuff we leave in files, folders, and SaaS silos can be pulled together and held to the standard of care we gave structured data. We are putting our own digital work under a microscope, and borrowing from fields that have managed their records rigorously for decades, like library science or intelligence analysis. Knowledge management tried this in the '90s, and the manual work it took to build and maintain created too much friction for it to scale. LLMs are taking most of that work away, which is why it's timely to give it a second chance.

When we optimize for humans, it often has benefits for LLMs too. Material that keeps its sources and its kind (a verbatim quote, a checked claim, a guess) can be fed into further LLM calls with less risk of hallucination. Loose AI-flavored prose, recycled from call to call, degrades quickly.

The old-fashioned field we need here is epistemology. In our case, it means tracking provenance and trust so that a written assertion can be relied on. Claims point to their sources, kept with enough context to survive re-use, arguments point to the claims they are built from, and both record who stands behind them.

Managing those connections is a whole layer that is not yet mainstream. This document describes what it would mean to build that layer.

¹ The scope is “knowledge work” outside of coding, which doesn't have a crisp name of its own. Coding is deliberately set aside because it has its own history and its own specific use of LLMs.

§ 1 What knowledge work needs in the LLM era

- **Epistemic types for text.** Mainstream tools file text by subject, format, author, and date. How far a piece of text can be trusted has been tracked only in niches, and by hand, in legal citators, in source grading for intelligence work, in peer review. What LLM-era work needs is filing by epistemic type, which is about where a piece came from, whether it was checked, and who stands behind it. A captured source, a verified fact, an asserted claim, an argument over claims, a position held, an open question, and a ruling are different things, and the tools need to tell them apart.
- **Grounding every piece of information.**
 - *Third party:* Every claim should point back to its sources, and the links should stay checkable so the claim can be re-verified later. Each piece of evidence (I call those “atoms”) should be one quote from one source, small enough to support a single point. Checking gets harder the more you care about context and intent (see the three kinds of check in section 2).
 - *First party:* We need to know whether a person or an AI tool produced each piece of work, at a finer grain than today’s tools record. This covers drafted text and recorded judgments, like a confidence score, a source rated unreliable, a claim marked verified, whether the operator (the person running the setup) or the LLM made the call. Without a record of authorship, you can’t tell a year later which verdicts you adjudicated from the ones the LLM filled in for you.
- **Recording human judgment.** More information means more decisions to make, more claims to accept or reject. Not all of those can fit in your head, and what is not written down gets re-derived or contradicted later. Of everything on this list this is the newest practice. Few people think about it, and there is next to no tooling for it.
- **Deliberate human attention.** Human attention is finite, and knowledge work is deep work that needs long stretches of it. The tools should protect that. They should bring the material the task needs, in the form the operator needs it, and they should never create chores of their own (correcting the AI’s mistakes, re-checking what has already been verified).
- **Governance of AI itself.** “AI governance” covers at least three layers.
 - The operating layer: what the AI does at what level of autonomy, who signs off and owns the result, and what budget it gets. This is what is meant in most business conversations.
 - The data layer: classification, access, retention. This is often presented, correctly, as the requirement to enable AI and how its output is marshaled.
 - The workspace layer: files, folders, workflows, and lifecycle states, the working environment itself treated as governed data. Out of the box, LLMs are prone to document sprawl and to jumping the gun by generating all kinds of content you didn’t specify, and LLM wrappers and SaaS products add restrictions of their own. The tools have to constrain what the LLM produces and still let the operator define the taxonomy of

documents and workflows.

This last layer is missing from the AI-governance conversation, and outside regulated industries, from what companies are doing.

- **Working with others.** Everything above has been tested within a one-person setup, and how it gets shared is still an open question. Two people need semantic interoperability, which means a source, a verified claim, or a ruling must mean the same thing to both of them. To go a little further, there is also what the literature calls pragmatic interoperability, which is about applying the same standard of care to the data that gets captured. It's easy to convert formats, but the meaning of the data at capture time does need to be agreed on beforehand.

§ 2 Why this hasn't been solved, and what to build on

- **Diagnosing the missing pieces.**
 - *Primitives need to be mechanical for a working discipline to form.* Code review formed around diff, continuous integration around tests. Knowledge work has no diff for arguments, no test that runs on claims as a document grows, no lint for provenance, so the corresponding disciplines haven't crystallized.
 - *It's tempting to aim for goals that are too broad.* Primitives for knowledge work in general have failed again and again: the space is challenging for one tool to be both specific enough to be efficient and open enough for a great variety of workflows. The SaaS products that succeeded picked a use case along with workflow constraints, and frustrate users for that very reason.
 - *A lot of manual work has traditionally been involved.* Building, curating, and then using those systems is manual work, and that is broadly the state of the "personal knowledge management" field. Systems of record show the same failure at company scale. A CRM has to be maintained by hand and decays because of it. The enterprise wiki (Confluence, SharePoint) goes stale the same way, until people have had enough and either go for a big clean-up or drop it entirely.
- **Building primitives, disciplines, and surfaces.** Software engineering offers a useful way to see what has to be built.
 - *Mechanical primitives:* capturing a quote and exactly where it sits in its source, checking that a quote says what we claim it says, checking a document against the rules for its kind, comparing two versions of a text, keeping a fingerprint of every file as it was sent.
 - *Disciplines built on them:* the source is captured at first retrieval, a quote is not cited until it has been checked against its source, only the operator moves a claim's standing.
 - *Surfaces where the work happens:* the editor, the terminal, the chat window, a reading and markup application, the shipped PDF.

After researching the field, I found plenty of tools and patterns for "memory", "context",

and knowledge graphs. Most of them do a fairly old thing (storing, linking, and retrieving text), and few have moved beyond what was done before LLMs. What is new is the economics of having LLMs fill them and read them, which removes the labor that sank knowledge management the first time around. What none of them add is a primitive for checked provenance, for checking claims, or for recording what you stand behind. Built on the right primitives, the records, the disciplines, and the surfaces all outlast any particular LLM, and each new LLM makes them more useful.

- **Performing three kinds of check.** A syntactic check looks only at the text itself, not its meaning: “is this quote present in that source?”. A semantic check reads its meaning: “does this claim follow from that quote?”. A pragmatic check depends on the interpreter and the purpose: “is this the right argument for this reader?”. Syntactic checks mechanize reliably, semantic checks partially, pragmatic checks least. Handing unstructured prose to the LLM turns the output into one woolly ball of automation where nothing in particular is being checked.

§ 3 The methods and tools I am building

Each piece below is a concept with its own terms, implemented as methods and tools. Each can be adopted on its own, and they are stronger together: claims lean on atoms, and the surface is only as good as the material under it. Keeping them separable is a design decision. It keeps the whole from becoming a single system nobody adopts, and it follows the Unix habit of small tools that each do one thing and compose.

- **Grounding: a capture format and its verification tooling.** An atom carries everything you need to use a piece of evidence away from its source. A verbatim quote, a link to the original source, metadata about the retrieval and its date, and the context needed to support that quote. The quote is precise enough to be checked mechanically against the source, and the statement the atom makes about the quote is checked for entailment, whether the quote actually supports it (see the glossary in section 5).
- **First-party attribution: a record of who wrote what.** Today’s tools don’t record whether something was written by a human or an AI tool. A Google Doc names the authors of a text but nothing finer, and text written with Gemini is filed under the human’s name. Git records every change, but at a coarser grain, and without recording AI use. The exception is code, where Cursor Blame now marks which lines an AI wrote and with which model. The nearest thing for prose is Grammarly’s authorship report, a per-document sidecar inside its own app. Nothing keeps that record across a working corpus, so I built one: a shadow record that commits at every file save with who made the change, person or AI.
- **Endorsement: a dated registry of standing.** Claims store what you think of them. Each claim carries a standing (proposed, held, contested, or retired), and only the operator moves it. Each move is dated in the file’s history, so a withdrawal is a recorded event rather than a deletion. The rest of the corpus links to the claim rather than restating it,

and arguments and documents are assembled from claims, which is a topic for another doc.

- **The governance layer: document contracts, committed and linted.** This is the workspace layer of governance from section 1. So much is implied when you work on a kind of document, and an LLM needs it spelled out. Humans benefit from that too, from being specific about what a document is for and what shape it takes. Optimizing your workflow to fit an LLM turns out to sharpen your own thinking about the kinds of documents you work on. I had underestimated the amount of constraints I needed to impose on LLM use and on my own workflows. Again and again, I had an artifact that was produced quickly but needed a lot of work to get into a good final shape. The solution was to add more governance and it is where most of the value turned out to be.
- **The interaction surface: a native UI.** Tools of my own for reading, annotating, navigating, and steering the corpus, by screen and by voice. Plain text can manage all of it, but it is missing many opportunities for better UX. Speech-to-text became ubiquitous in 2026 and was a major productivity boost, but beyond that little exists that is more sophisticated than a chatbot, and every power user of Claude Code has felt the limitations.

What I do not know is how this breaks at larger scale. My system holds 500+ atoms, with one operator, in about six months.

§ 4 Where this is going

THREE CURRENTS CONVERGE

Three currents are converging and give an idea of the broader arc:

- **Data analysis keeps chasing data capture.** The web was meant to be semantic but what accumulated was unstructured. Big data, with machine learning, was built to make sense of it after the fact. Now the knowledge sits in both the LLMs' weights and what they pull from the web, and analysis is again racing to catch up.
- **Knowledge management is coming back to the forefront.** Search engines, full-text databases, and document management systems all manage text well: stored, indexed, versioned, retrieved. Knowledge is text you can check, combine, and stand behind. Pre-LLMs, those properties were largely invisible to software. Knowledge engineering in the 1980s came at it from the formal side, and its descendant is alive in Palantir's hand-built ontologies, which work for operational data. Knowledge management in the 1990s came at it from the people side, and its descendants are the company-brain startups.
- **A new iteration of Agile.** When iterating on software became faster, new methodologies were championed: this was the start of Agile, with short iterations, continuous integration, and refactoring. LLMs now make it possible to operate quickly on knowledge. If that holds, knowledge work gets its own round of new (or re-newed) methodologies, probably inspired by the original Agile field.

THE COMPANY-BRAIN TREND

- **“Brain” describes no mechanism.** “Company brain” was a Y Combinator request for startups for its Summer 2026 batch, a cluster of startups use the name, and the business press is pitching “enterprise brain” at the enterprise vendors. Unpack any of them and what ships today is retrieval over a shared corpus, permission-aware, with agents on top and, in a few, a typed graph or a synthesis layer. The word anthropomorphizes and specifies nothing.
- **The records under retrieval are missing.** The unit in those products is the page or the chunk, not the claim, which is why the rest is absent. Citations point at a page, or highlight a passage while the answer is on screen, and where a quote is kept, nothing checks it (grounding, present but weak). Nothing records what the organization stands behind, claim by claim and including withdrawal (endorsement; Notion’s verified pages and Guru’s cards mark whole pages and let them lapse). Nothing records who wrote what, human or AI, below the grain of a whole page (attribution). Those three are what this document builds.
- **The industry is racing to the top of that stack.** Agentic AI is being marketed while much of what sits underneath it is underserved. The word is overloaded by now, and non-deterministic workflows need a quality of structure that hasn’t had time to emerge outside of coding.

§ 5 Terms used in this document

- **Epistemic type** (section 1): the type of a piece of text by what stands behind it (a source, a check, a person’s standing, a ruling) rather than by its subject or format.
- **Claim** (throughout): a proposition put forward as true, resting on one or more atoms and carrying a standing. Arguments and documents are assembled from claims.
- **Atom** (sections 1 and 3): one quote from one source, captured with enough context to support a single point.
- **Endorsement** (sections 1 and 3): the standing a person gives a claim, held or withdrawn, with the date of each change.
- **Semantic and pragmatic interoperability** (section 1): two systems are semantically interoperable when what they exchange means the same on both sides, not just shares a format, and pragmatically interoperable when both apply the same standard when they capture and use it.
- **Syntactic / semantic / pragmatic** (section 2): “pragmatic” carries its linguistic sense here (depending on the interpreter and the purpose), not the everyday sense of practical or sensible.
- **Entailment** (section 3): whether the quote actually supports the claim that cites it: would a reader holding only the quote, with its surrounding context, accept the claim?